현대 AI 개론

PART V. 검색과 지식

외부 지식을 AI에 연결하는 기술

RAG에서 개인 지식 관리까지

공냥이

2026

목차

17 검색해서 넣어주면 된다는 믿음의 현실 3

RAG는 두 가지 구조적 결함에서 태어났다 3

검색이 틀리면 모든 것이 무너진다 3

맞는 문서를 가져왔는데 모델이 무시한다 4

청킹이 문맥을 끊는 순간 4

롱컨텍스트가 RAG를 대체하지 못하는 이유 5

하이브리드 검색과 재순위화가 표준이 된 이유 6

에이전틱 RAG: 한 번 검색으로 부족하면 다시 찾는다 6

RAG를 쓸 때와 쓰지 말아야 할 때 7

18 지식이 구조화되지 않으면 AI는 무력하다 8

컨텍스트 엔지니어링은 지식 관리 없이 불가능하다 8

Karpathy LLM Wiki 패턴: 원본과 가공물을 분리한다 8

크로스링크가 지식의 가치를 만든다 9

구조화된 지식이 RAG의 품질을 결정한다 9

개인 지식 관리: 도구보다 원칙이 먼저다 10

AI가 노트 사이의 연결을 자동으로 찾는다 10

조직 지식 관리: 퇴사자가 나가면 지식이 사라진다 11

제2의 뇌는 기억이 아니라 연결이다 11

CHAPTER 17

검색해서 넣어주면 된다는 믿음의 현실

LLM은 학습 시점 이후의 세계를 모른다. 외부 정보를 검색해서 컨텍스트에 넣어주면 이 문제가 풀린다는 아이디어가 RAG다. 이 아이디어는 반만 맞았다.

RAG는 두 가지 구조적 결함에서 태어났다

도서관을 떠올려보자. 사서가 아무리 박식해도, 어제 출간된 책의 내용은 알 수 없다. 그 책을 직접 찾아서 읽어야 한다. LLM도 마찬가지다. 아무리 거대한 모델이라도 학습 데이터 컷오프 이후의 정보는 갖고 있지 않다.

LLM에는 두 가지 구조적 결함이 있다. 첫째, 학습 데이터 컷오프다. 2024년까지 학습한 모델은 2025년의 사건을 모른다. 둘째, 환각이다. 모르는 것을 모른다고 말하지 않고 그럴듯하게 지어낸다. 이 두 문제가 결합되면 위험해진다. 모델이 최신 정보를 모르면서도 아는 척 답변하는 상황이 벌어지기 때문이다.

2020년, Meta AI의 Patrick Lewis 팀이 이 문제에 대한 해법을 제시했다. RAG(Retrieval-Augmented Generation, 검색 증강 생성)라는 이름의 이 접근법은 생성 모델에 검색 모듈을 결합한 것이다. 이름을 풀어보면 의미가 명확하다. Retrieval은 외부 지식 저장소에서 관련 문서를 가져오는 검색 단계다. Augmented는 그 검색 결과로 모델의 입력을 보강한다는 뜻이다. Generation은 보강된 컨텍스트를 바탕으로 답변을 생성하는 최종 단계다. Lewis 팀은 이 세 단계를 하나의 파이프라인으로 묶어서, 모델이 자신의 파라미터에 저장되지 않은 지식도 활용할 수 있게 만들었다. 이 논문이 2020년 NeurIPS에 발표된 이후, RAG는 LLM 활용의 사실상 표준 패턴이 됐다.

작동 원리는 단순하다. 사용자가 질문하면, 먼저 외부 데이터베이스에서 관련 문서를 검색한다. 검색된 문서를 컨텍스트에 넣어준다. 모델은 그 문서를 참조해서 답변을 생성한다. 사서에게 질문하기 전에 관련 책을 먼저 꺼내서 펼쳐놓는 것과 같다.

단순해 보인다. 그러나 이 단순한 파이프라인의 각 단계에서 실패가 발생한다.

검색이 틀리면 모든 것이 무너진다

RAG의 첫 번째 취약점은 검색 단계에 있다. 검색이 잘못된 문서를 가져오면, 그 뒤의 모든 과정이 오염된다.

양자컴퓨팅의 최신 동향을 알려줘라고 질문했다고 하자. 벡터 검색이 2022년 블로그 포스트를 가져왔다. 최신이 아니다. 더 위험한 경우도 있다. 질문에 **양자**라는 단어가 들어 있어서 양자역학 입문 문서를 가져오는 것이다. 양자컴퓨팅이 아니라 양자역학이다. 검색이 의미가 아니라 단어의 유사성에 속은 셈이다.

이 문제가 왜 심각한지를 이해하려면 벡터 검색의 작동 방식을 알아야 한다. 벡터 검색은 텍스트를 고차원 벡터로 변환한 뒤, 벡터 간 거리가 가까운 것을 **유사하다**고 판단한다. 문제는 의미적 유사성과 벡터 거리가 항상 일치하지 않는다는 점이다. **사과**와 **배**의 벡터 거리가 가까운 것은 합리적이지만, **Apple**이 과일인지 회사인지는 벡터만으로 구분하기 어렵다.

> **검색이 실패하면 RAG 전체가 실패한다. 아무리 좋은 LLM이라도 잘못된 소스를 받으면 잘못된 답을 내놓는다. 쓰레기가 들어가면 쓰레기가 나온다.**

맞는 문서를 가져왔는데 모델이 무시한다

두 번째 실패 지점은 생성 단계에 있다. 관련 문서를 정확히 찾아서 컨텍스트에 넣어줬는데, 모델이 그 문서 대신 자신의 사전학습 지식을 우선해서 답하는 경우가 있다.

Huang 등의 서베이 논문이 이 현상을 체계적으로 정리했다. RAG만으로는 불충분하다. 모델이 잘못된 소스를 수용하거나, 맞는 소스를 무시할 수 있다. 이것은 충실성 환각(Faithfulness Hallucination)의 변종이다. 모델이 컨텍스트에 제공된 정보보다 자신의 내부 지식을 신뢰하는 현상이다.

왜 이런 일이 벌어지는가. 모델의 사전학습 지식과 검색된 문서의 내용이 충돌할 때, 모델은 둘 중 하나를 선택해야 한다. 사전학습 과정에서 수십억 토큰에 걸쳐 학습한 패턴은 강력하다. 컨텍스트에 넣어준 몇 천 토큰의 문서가 그 패턴을 항상 이기지는 못한다. 시험 전날 벼락치기한 내용보다 평소 믿고 있던 상식이 더 강하게 작용하는 것과 같다.

이 문제는 특히 모델의 학습 데이터에 강하게 각인된 정보와 충돌할 때 심해진다. 모델이 **확신하는** 정보일수록 외부 소스를 무시할 확률이 높아진다.

청킹이 문맥을 끊는 순간

세 번째 실패 지점은 문서를 조각내는 과정에 있다. RAG는 문서를 작은 조각, 즉 청크(chunk)로 나눠서 벡터 데이터베이스에 저장한다. 문서 전체를 하나의 벡터로 만들면 검색 정밀도가 떨어지기 때문이다.

문제는 이 과정에서 문맥이 끊긴다는 점이다. 구체적 예를 보자.

구분	내용

원본	2024년 매출은 150억이며 전년 대비 23% 증가다. 이 성장의 주된 원인은 신규 B2B 채널 확대다
청크 1	2024년 매출은 150억이며 전년 대비 23% 증가다
청크 2	이 성장의 주된 원인은 신규 B2B 채널 확대다

매출 성장 원인이 뭐야라고 질문하면 검색은 청크 2만 가져온다. 150억이라는 수치를 모른 채 원인만 답하게 된다. 문서를 자르는 과정에서 문맥이 분리된 것이다. 이 현상은 표, 코드, 리스트 같은 구조화된 콘텐츠에서 특히 심각하다. 표의 헤더와 데이터가 다른 청크에 배정되면 의미가 완전히 소실된다.

청킹 전략을 개선하는 연구가 계속되고 있다. 문장 단위가 아닌 의미 단위 청킹, 계층적 청킹, 문서 구조를 보존하는 청킹 등이 제안됐다. 그러나 완벽한 청킹은 존재하지 않는다. 문서를 자르는 순간 일부 문맥은 반드시 손실된다.

롱컨텍스트가 RAG를 대체하지 못하는 이유

많은 사람이 묻는다. 컨텍스트 윈도우가 1M 토큰이면 문서를 통째로 넣으면 되지 않느냐고. 맞다. 그것이 롱컨텍스트(Long-Context) 접근이다. 청킹도 필요 없고, 검색 실패도 없다.

Li 등의 연구가 이 질문에 답했다. 결론은 이분법이 틀렸다는 것이다.

기준	롱컨텍스트	RAG
정확도	리소스 충분 시 평균적으로 우위	검색 품질에 의존
비용	토큰 수에 비례해 급증	검색된 청크만 비용 발생
지연 시간	컨텍스트가 길수록 느림	상대적으로 일정
최적 전략	—	—

리소스가 충분할 때는 롱컨텍스트가 평균적으로 나았다. 그러나 RAG는 비용 우위를 유지했다. 100K 토큰 문서를 매번 통째로 넣는 것과, 관련 청크 5개(약 2K 토큰)만 넣는 것의 비용 차이는 50배다. 사용량이 많은 프로덕션 환경에서 이 차이는 결정적이다.

가장 주목할 결과는 세 번째 행, 최적 전략이다. Li 등이 제안한 Self-Route 하이브리드 방식에서 모델이 스스로 판단한다. 이 질문은 전체 문서가 필요한가, 아니면 검색으로 충분한가. 이 적응적 판단이 어느 한쪽만 쓰는 것보다 일관되게 나은 결과를 보였다.

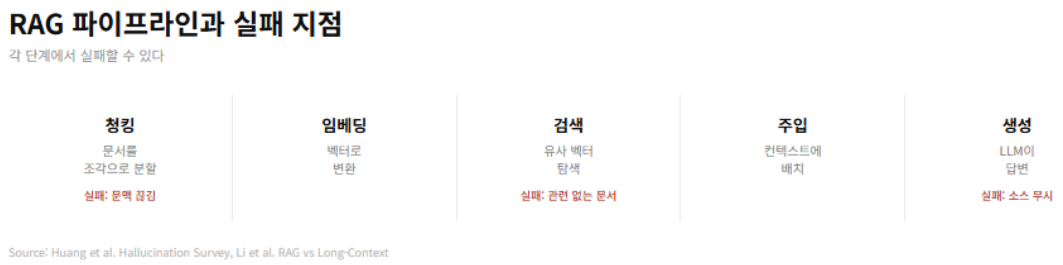

그림 1: RAG 파이프라인의 구조. 질문에서 검색, 컨텍스트 주입, 생성까지의 흐름과 각 단계의 실패 지점을 보여준다.

이 그림에서 주목할 것은 화살표 사이의 각 단계다. 검색, 청킹, 생성 어디에서든 실패가 발생할 수 있으며, 한 단계의 실패가 전체 파이프라인을 오염시킨다.

하이브리드 검색과 재순위화가 표준이 된 이유

RAG의 검색 품질을 끌어올리는 방법은 두 가지 검색을 병행하는 것이다. 벡터 검색은 의미적 유사성에 강하지만 정확한 용어 매칭에 약하다. 키워드 검색(BM25)은 정확한 용어 매칭에 강하지만 의미적 유사성을 포착하지 못한다. 둘을 결합하면 각자의 약점을 보완할 수 있다.

이 결합만으로는 부족하다. 두 검색의 결과를 병합한 뒤, 재순위화(Reranking) 모델이 결과를 다시 정렬한다. 재순위화 모델은 초벌 검색보다 훨씬 정교한 모델이다. 속도는 느리지만 정확도가 높다. 초벌 검색이 100개 후보를 빠르게 추리고, 재순위화가 그중 상위 5개를 정밀하게 골라내는 구조다.

서점 직원이 먼저 해당 코너에서 관련 책 10권을 뽑아놓고, 전문가가 그중에서 가장 적합한 3권을 골라주는 것과 같다. 2026년 현재 이 하이브리드 검색 + 재순위화 조합이 RAG 파이프라인의 사실상 표준이다.

에이전틱 RAG: 한 번 검색으로 부족하면 다시 찾는다

2026년의 RAG는 한 번 검색하고 끝내지 않는다. 에이전틱 RAG(Agentic RAG)는 검색 결과를 평가하고, 부족하다고 판단하면 다른 키워드로, 다른 소스에서 추가 검색을 수행한다.

방식	작동 흐름
전통적 RAG	검색 1회 → 답변 생성 (원샷)
에이전틱 RAG	검색 → 답변 초안 → 부족 판단 → 재검색 → 보강 → 최종 답변

이 구조는 PART IV에서 다뤘던 에이전트 루프의 Gather 단계에 RAG가 들어간 것이다. 에이전트가 자신의 답변을 스스로 검증하고, 근거가 부족하면 추가 정보를 수집하는 과정이다. 한 번의 검색에 운을 맡기는 것이 아니라, 반복적으로 정보를 모으고 검증하는 것이다.

이것이 RAG의 현재 진화 방향이다. 단순한 검색-생성 파이프라인에서 자율적 정보 수집 루프로의 전환이다.

> **RAG는 죽지 않았다. 컨텍스트 엔지니어링의 한 도구로 흡수됐다. Lance Martin이 RAG를 Select 축의 핵심 기법으로 재정의한 것이 이 전환을 정확히 짚는다.**

RAG를 쓸 때와 쓰지 말아야 할 때

모든 문제에 RAG가 답은 아니다. 어떤 경우에 RAG가 적합하고, 어떤 경우에 다른 접근이 나은지를 구분해야 한다.

상황	적합한 접근	이유
최신 정보가 필요	RAG	학습 데이터 컷오프를 우회
내부 문서 기반 답변	RAG	모델이 학습하지 않은 도메인 지식
비용이 제약	RAG	필요한 청크만 넣어 토큰 비용 절감
높은 정확도가 필수	롱컨텍스트 또는 파인튜닝	검색 실패 위험을 줄임
도메인 고정	파인튜닝	반복적 검색보다 내재화가 효율적

이 표에서 주목할 것은 세 번째와 네 번째 행의 대비다. 비용을 줄이고 싶으면 RAG가 유리하지만, 정확도가 생명인 의료나 법률 영역에서는 검색 실패 리스크를 감수하기 어렵다. 한 기법이 모든 상황을 지배하는 것이 아니라, 상황에 따른 선택의 문제인 셈이다.

다음 글에서는 이 검색된 지식을 어떻게 구조화하고 관리하는지를 다룬다. 지식이 구조화되지 않으면 검색의 품질도 올라가지 않기 때문이다.

Source: Lewis et al. (NeurIPS) 2020; Huang et al. — Hallucination Survey; Li et al. — Long-Context vs RAG, Self-Route; Lance Martin — RAG as Select axis

CHAPTER 18

지식이 구조화되지 않으면 AI는 무력하다

AI 시대에 가장 과소평가된 역량은 지식 관리다. 정보가 어디에 있는지 모르면 검색도, 컨텍스트 엔지니어링도, 에이전트도 제대로 작동하지 않는다. 구조화된 지식이 AI의 연료다.

컨텍스트 엔지니어링은 지식 관리 없이 불가능하다

냉장고를 떠올려보자. 재료가 아무렇게나 쌓여 있으면 요리할 때 필요한 것을 찾지 못한다. 반면 재료가 종류별로 정리되어 있으면 레시피를 보고 곧바로 꺼낼 수 있다. AI의 컨텍스트 엔지니어링도 마찬가지다.

PART III에서 컨텍스트 엔지니어링의 핵심이 **다음 스텝에 딱 맞는 정보를 넣는 것**이라고 했다. 그런데 그 정보가 어디에 있는지 모르면 넣을 수 없다. Notion에 있는지, Google Drive에 있는지, 누군가의 머릿속에 있는지, Slack 대화 어딘가에 묻혀 있는지 모른다면 컨텍스트 엔지니어링 자체가 불가능하다.

이 문제는 AI 이전에도 있었다. 그러나 AI 이전에는 사람이 기억과 경험으로 메꿀 수 있었다. **그건 김 대리한테 물어봐**가 통했다. AI는 김 대리의 기억에 접근할 수 없다. 명시적으로 구조화된 지식만 활용할 수 있다. 지식 관리가 **좋으면 좋고 없어도 그만**이던 시대는 끝났다. 2026년 현재 지식 관리는 AI 활용의 전제 조건이다.

Karpathy LLM Wiki 패턴: 원본과 가공물을 분리한다

이 책의 소스인 gongnyang-wiki는 3,262개 소스 문서에서 만들어졌다. 이 위키가 따르는 구조가 **Karpathy LLM Wiki 패턴**이다. Andrej Karpathy가 제안한 이 패턴의 핵심은 원자료와 가공물의 분리다.

계층	역할	규칙
raw/ (원자료)	논문, 기사, 블로그, 문서를 원본 그대로 저장	절대 수정하지 않는다

wiki/ (가공물)	raw를 기반으로 요약, 개념, 엔티티, 비교를 생성	한국어로 통일, 크로스링크로 연결

왜 이 분리가 중요한가. 원본을 수정하면 출처를 잃는다. 해석이 바뀔 때 원본까지 바뀌면 나중에 어디서 왜 틀렸는지 추적할 수 없다. 법정에서 원본 증거를 보존하는 것과 같은 원리다. 원본은 불변이고, 해석만 갱신된다.

이 구조가 앞 글에서 다룬 RAG의 한계를 다르게 접근한다. RAG는 질문 시점에 실시간으로 관련 문서를 검색한다. 검색이 실패하면 답변도 실패한다. LLM Wiki 패턴은 미리 구조화해놓는다. 어디에 무엇이 있는지 이미 알고 있으므로 검색에 의존하지 않는다. 검색이 아니라 탐색이다.

크로스링크가 지식의 가치를 만든다

낱장의 메모는 그 자체로 가치가 제한적이다. 메모와 메모 사이의 연결이 지식의 가치를 만든다. 한 장의 카드에 적힌 사실은 정보다. 그 카드가 다른 카드들과 연결될 때 지식이 된다.

크로스링크는 개념 간의 관계를 명시적으로 표현하는 것이다. 이 책을 쓸 때의 실제 과정을 보자. 프롬프트 엔지니어링의 한계를 다루는 글을 쓸 때, wiki/concepts/프롬프트-엔지니어링.md에서 25개 소스가 매핑되고, wiki/concepts/컨텍스트-엔지니어링.md에서 31개 소스가 매핑된다. 관련 요약 5개를 로드하고, 원본에서 인용을 확인한다.

이 과정에서 검색은 발생하지 않았다. 구조가 이미 경로를 알려주기 때문이다. 도서관에서 책을 찾을 때 서가 번호를 아는 것과 모르는 것의 차이다. 분류 체계가 있으면 바로 찾아가고, 없으면 하나하나 뒤져야 한다.

크로스링크의 또 다른 가치는 빈 곳을 드러내는 것이다. 연결이 적은 개념은 아직 충분히 탐구되지 않은 영역이다. 연결이 많은 개념은 핵심 허브다. 이 지도가 없으면 어디를 더 파야 하는지조차 모른다.

> **지식의 가치는 양이 아니라 연결에 있다. 1,000개의 고립된 메모보다 100개의 연결된 메모가 더 강력하다.**

구조화된 지식이 RAG의 품질을 결정한다

앞 글에서 RAG의 세 가지 실패 지점을 다뤘다. 검색 실패, 소스 무시, 청킹 손실. 이 세 가지 모두 지식이 구조화되지 않았을 때 악화된다.

검색 실패를 예로 들어보자. 같은 개념을 어떤 문서에서는 **RAG**로, 다른 문서에서는 **검색 증강 생성**으로, 또 다른 문서에서는 **retrieval-augmented generation**으로 표기하면 벡터 검색의 정확도가 떨어진다. 위키 패턴에서는 가공물 계층에서 용어를 통일한다. 하나의 개념에 하나의 정규 이름을 부여하고, 나머지는 해당 정규 이름으로 연결한다.

청킹 손실도 마찬가지다. 구조화되지 않은 긴 문서를 기계적으로 자르면 문맥이 끊긴다. 위키 패턴에서는 개념 단위로 문서가 이미 분리되어 있다. 하나의 파일이 하나의 개념을 다루므로, 파일 자체가 의미 있는 청크다. 기계적 청킹이 필요 없어지는 셈이다.

지식 구조화가 RAG의 전처리 역할을 한다고 볼 수 있다. RAG의 검색 품질은 검색 알고리즘만으로 결정되지 않는다. 검색 대상인 지식의 구조가 품질의 상한선을 결정한다.

개인 지식 관리: 도구보다 원칙이 먼저다

나도 이런 시스템을 만들고 싶다는 사람에게 도구부터 추천하면 안 된다. 도구는 바뀔 수 있지만 원칙은 남기 때문이다.

원칙	설명
원본 보존	수집한 자료는 원본 그대로 저장한다. 해석은 별도 파일에 분리한다
크로스링크	모든 노트는 최소 2개 이상의 다른 노트와 연결한다
주기적 정리	수집만 하고 정리하지 않으면 쓰레기장이 된다. 주 1회 이상 정리한다
AI 활용	요약, 태깅, 연결 제안을 AI에게 맡긴다. 사람은 판단에 집중한다

도구는 이 원칙 위에서 선택한다. 2026년 현재 대표적 도구 세 가지를 비교하면 다음과 같다. Obsidian은 로컬 마크다운 기반으로 위키링크를 지원하며, 데이터가 사용자 컴퓨터에 남는다. Notion은 클라우드 기반으로 팀 협업에 강하다. Logseq는 개요(outliner) 기반으로 일기와 메모에 적합하다.

어떤 도구를 선택하든 네 가지 원칙을 지킬 수 있으면 된다. 도구에 갇히면 안 된다.

AI가 노트 사이의 연결을 자동으로 찾는다

2025 2026년에 폭발적으로 성장한 영역이 있다. AI 기반 노트 자동 연결이다.

새 노트를 작성하면 AI가 기존 노트와의 관련성을 분석한다. **이 노트는 컨텍스트 엔지니어링, 하네스, RAG와 관련됩니다**라고 자동으로 위키링크를 제안한다. 수천 개 노트가 있어도 관련 노트를 자동으로 찾아서 연결한다.

이 자동 연결이 왜 중요한지를 예로 들어보자. 1,000개의 노트를 가진 사람이 새 노트를 하나 추가했다. 사람이 직접 연결하려면 1,000개를 전부 기억하고 있어야 한다. 현실적으로 불가능하다. AI는 1,000개 노트의 내용을 전부 비교해서 관련성이 높은 5 10개를 골라낸다. 사람이 놓치는 연결을 AI가 발견하는 셈이다.

Obsidian에 AI 플러그인을 연결하면 이 기능을 바로 쓸 수 있다. Notion도 2025년부터 AI 기반 연결 기능을 제공하기 시작했다. 도구가 아니라 **연결을 자동화하는 접근**이 핵심이다.

그림 2: KV-Cache와 지식 관리의 관계. 구조화된 지식은 불필요한 컨텍스트를 줄여 KV-Cache 비용을 절감한다.

이 그림에서 주목할 것은 지식 구조화와 비용의 관계다. 지식이 잘 정리되어 있으면 필요한 정보만 정확히 넣을 수 있고, 불필요한 토큰을 줄일 수 있다. 앞 글에서 다룬 KV-Cache 비용이 컨텍스트 길이에 비선형으로 증가한다는 점을 고려하면, 지식 관리는 성능만이 아니라 비용도 최적화하는 셈이다.

조직 지식 관리: 퇴사자가 나가면 지식이 사라진다

개인 수준에서 조직 수준으로 올라가면 문제가 더 심각해진다. 세 가지 전형적 상황이 있다. 신규 입사자가 **이 프로젝트 어떻게 돌아가요**라고 물으면 아무도 정리해놓은 사람이 없다. 6개월 전에 결정한 아키텍처의 이유를 아무도 기억하지 못한다. 퇴사자가 나가면 머릿속 지식이 함께 사라진다.

이 문제에 대한 AI 기반 해결책은 세 가지 계층의 결합이다.

계층	역할
구조화된 지식 베이스 (Wiki 패턴)	핵심 개념, 의사결정 이유, 프로세스를 체계적으로 정리
RAG (검색)	정리된 지식 베이스에서 질문에 맞는 정보를 검색
AI 에이전트 (자동 업데이트)	새 문서, 회의록, Slack 대화를 자동으로 지식 베이스에 반영

이 세 계층이 합쳐지면 **질문하면 조직의 모든 지식에서 답이 나오는** 시스템이 된다. 그러나 앞 글에서 다뤘듯이 RAG만으로는 부족하다. 구조화 없이 RAG를 적용하면 검색 품질이 떨어진다. 모든 문서를 벡터 데이터베이스에 넣었다고 해서 지식 관리가 되는 것은 아니다. 도서관에 책을 쌓아놓기만 하고 분류하지 않으면 도서관이 아니라 창고인 것과 같다.

제2의 뇌는 기억이 아니라 연결이다

제2의 뇌(Second Brain)라는 표현이 유행한 지 수년이 지났다. Tiago Forte의 책 이후 많은 사람이 지식 관리 시스템을 만들기 시작했다. 그러나 대부분은 수집에서 멈춘다. 수집만 하고 연결하지 않은 지식은 뇌가 아니라 하드디스크다.

진짜 뇌가 하는 일은 기억이 아니다. 연결이다. 신경세포 하나의 정보는 의미가 없다. 신경세포 간의 시냅스가 의미를 만든다. 지식 관리도 마찬가지다. 노트 하나의 가치는 제한적이다. 노트와 노트 사이의 링크가 가치를 만든다.

AI 시대의 제2의 뇌는 세 가지 특성을 갖는다. 첫째, 원본이 보존된다. 둘째, 개념 간의 연결이 명시적이다. 셋째, AI가 연결을 자동으로 발견하고 제안한다. 이 세 가지가 갖춰지면 지식은 시간이 갈수록 더 가치 있어진다. 새로운 정보가 추가될 때마다 기존 지식과의 연결이 늘어나기 때문이다.

> **지식 관리의 본질은 수집이 아니라 연결이다. AI는 이 연결을 자동화한다. 구조화된 지식 위에서 RAG도, 컨텍스트 엔지니어링도, 에이전트도 비로소 제 성능을 낸다.**

다음 파트에서는 시야를 넓혀서, 이 AI 기술들을 실제로 구동하는 생태계와 인프라를 살펴본다. 오픈소스 LLM 지형도부터 시작한다.

Source: Lewis et al. (NeurIPS) 2020; Karpathy LLM Wiki 패턴; Tiago Forte — Building a Second Brain; gongnyang-wiki 3,262 sources